Playwright- Screenshots, Videos, and Tracing (Trace Viewer)

🖼️ Screenshots in Playwright (with Pytest)

Screenshots help capture the state of your web page during test execution.

1. Capture a Screenshot

To take a screenshot of the current browser view:

```
page.screenshot(path='screenshot.png')
```

2. Capture a Full Page Screenshot

Takes a screenshot of the **entire scrollable area** of the web page:

```
page.screenshot(path='fullpage.png', full_page=True)
```

3. Capture a Specific Element

You can also capture a screenshot of a **particular element**:

```
page.locator('.header').screenshot(path='header.png')
```

⚙️ Configure Screenshots in pytest.ini

You can automatically control screenshot behavior from pytest.ini:

```
[pytest]
addopts = --screenshot=on
```

Option	Description
on	Always capture screenshots after tests
off	Disable screenshots
only-on-failure	Capture screenshots only when a test fails
on-first-failure	Capture only on the first failed test

🎬 Video Recording in Playwright (with Pytest)

Playwright can **record test execution videos** — useful for debugging visual issues or understanding test behavior.

Through Code

Example:

```
def test_video_recording(playwright):
    browser = playwright.chromium.launch()
    context = browser.new_context(record_video_dir="videos/")  # Folder to save videos
    page = context.new_page()
    page.goto("https://example.com")
    page.click("text=More information")
    context.close()  # Video is saved when context is closed
    browser.close()
```

⚙️ Configure Videos in pytest.ini

You can manage video recording behavior in pytest.ini:

[pytest]
addopts = --video=on

Option	Description
off	Disable video recording
on	Record all tests
retain-on-failure	Keep videos only for failed tests
on-first-retry	Record video only when a test is retried

🔍 Tracing and Trace Viewer

Playwright **Tracing** is a debugging tool that records everything happening during a test — including:

- User actions
- Network requests
- DOM snapshots
- Console logs

You can **replay and inspect** these traces later using **Trace Viewer**.

1. Start and Stop Tracing in Code

```
def test_tracing_example(playwright):
    browser = playwright.chromium.launch(headless=False)
    context = browser.new_context()
    # Start tracing with screenshots and snapshots
    context.tracing.start(screenshots=True, snapshots=True)
    page = context.new_page()
    page.goto("https://demowebshop.tricentis.com/")
    page.click("text=Login")
    # Stop tracing and save it as a zip file
    context.tracing.stop(path="trace.zip")
    browser.close()
```

⚙️ Enable Tracing in pytest.ini

You can enable tracing globally from **pytest.ini**:

```
[pytest]
addopts = --tracing=on
```

Option	Description
on	Record trace for each test
off	Disable tracing (default)
retain-on-failure	Keep traces only for failed tests

Enable Tracing from Command Line

You can also control tracing through CLI:

```
pytest --tracing=on
```

🧭 Viewing the Trace File (trace.zip)

Once tracing is enabled, Playwright generates a .zip file that contains the test trace.
You can view this trace in **three ways**:

✅ Option 1 — Command Line

```
playwright show-trace trace.zip
```

🌐 Option 2 — Online Viewer

Visit: https://trace.playwright.dev/
Then drag and drop your trace.zip file.

Flaky Tests and Retries in Pytest with Playwright

What Are Flaky Tests?

Flaky tests **sometimes pass and sometimes fail** even when the code hasn't changed — often due to:

- Slow network or animations
- Dynamic page elements
- Timing issues

Such tests make results unreliable.

✅ Playwright Retries (Pytest Style)

You can retry failed tests automatically in **pytest** using the **pytest-rerunfailures** plugin.

1. Install Plugin

```
pip install pytest-rerunfailures
```

2. Use CLI command

```
pytest prep/test_flaky.py --reruns 3 --reruns-delay 2
```

→ This retries failed tests **up to 3 times** with a **2-second delay** between each retry.

Example Scenario

Attempt	Result	Outcome
1	❌ Failed	Retry triggered
2	✅ Passed	Marked as flaky